```
builder.Services.ConfigureApplicationCookie(options =>
{
    options.AccessDeniedPath = "/identity/account/access-denied";
});
```

The declarative approach to authorizing an endpoint by decorating the `PageModel` with an attribute is quick and easy, but if you have a number of pages to protect against anonymous users, the only way to check that you have applied the attribute to the relevant `PageModel`s is to look at each file individually. If you want to protect the contents of an entire folder, you must remember to add the attribute to each and every page in the folder, including new ones that get added at some stage in the future. One way to mitigate this is to declare a class that derives from `PageModel` and apply the `Authorize` attribute to it and then get all the pages in the folder to inherit from it. The following illustrates how you can achieve this with a class named `AdminPageModel`.

Listing 10.4 Using a `BasePage` to restrict access to derivatives

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace CityBreaks.Pages
{
    [Authorize]
    public class AdminPageModel : PageModel
    {
    }
}
```

When you create a page that needs to be protected, you simply change the type it inherits from, so it derives from `AdminPageModel` and is automatically protected by the `Authorize` attribute:

```
public class CreateModel : AdminPageModel
```